

Proyecto 3 IA: Juegos Adversarios

Alina T. Carías (22539), Ignacio Méndez (22613), Ariela Mishaan (22052), Diego Soto (22737), *Ingeniería en Ciencia de los Datos, UVG*

Abstract—This project presents the development of an intelligent Othello-playing system using different adversarial search algorithms. Agents based on Alpha-Beta Pruning, Expectimax, and Monte Carlo Tree Search (MCTS) were implemented together with an interactive graphical interface capable of simulating matches between humans and intelligent agents. The experimental results show that Alpha-Beta provides strong and consistent gameplay due to heuristic evaluation functions, whereas MCTS demonstrates advantages in strategic exploration and Expectimax achieves intermediate performance depending on search depth.

Resumen— En este proyecto se desarrolló un sistema inteligente para jugar Othello utilizando distintos algoritmos de búsqueda adversaria. Se implementaron agentes basados en Alpha-Beta Pruning, Expectimax y Monte Carlo Tree Search (MCTS), junto con una interfaz visual interactiva para simular partidas entre humanos y agentes inteligentes. Los resultados obtenidos muestran que Alpha-Beta ofrece un comportamiento sólido y consistente gracias al uso de heurísticas posicionales, mientras que MCTS presenta ventajas en exploración estratégica y Expectimax exhibe un desempeño intermedio dependiendo de la profundidad de búsqueda.

Palabras clave—Othello, Alpha-Beta, Minimax, MCTS, Expectimax, Juegos Adversarios, IA

I. INTRODUCCIÓN

Los juegos adversarios constituyen uno de los problemas clásicos de la inteligencia artificial debido a la necesidad de modelar decisiones óptimas en entornos competitivos. Algoritmos como Minimax y Alpha-Beta han sido utilizados históricamente para resolver juegos deterministas de información perfecta, mientras que técnicas más modernas como Monte Carlo Tree Search permiten explorar espacios de búsqueda complejos mediante simulaciones probabilísticas.

En este proyecto se desarrolló un motor de juego para Othello, un juego de estrategia de suma cero donde dos jugadores compiten por controlar la mayor cantidad de fichas en un tablero de 8×8 . El sistema implementado permite enfrentar distintos agentes inteligentes y analizar su desempeño utilizando métricas cuantitativas.

El propósito principal fue comparar el comportamiento de diferentes algoritmos de búsqueda adversaria bajo restricciones de tiempo y evaluar sus fortalezas y debilidades en distintas situaciones del juego.

II. MARCO TEÓRICO

A. Juegos adversarios

Los juegos adversarios representan una de las áreas clásicas de estudio dentro de la inteligencia artificial, ya que permiten analizar cómo un agente puede tomar decisiones óptimas en entornos competitivos. En este tipo de problemas, dos o más jugadores poseen objetivos opuestos: mientras un jugador intenta maximizar su beneficio, el adversario busca minimizarlo. Muchos de estos juegos, como Othello, ajedrez o damas, son considerados juegos de suma cero y de información perfecta, debido a que toda la información del estado del juego está disponible para ambos participantes y no existen elementos ocultos ni aleatorios durante la partida (GeeksForGeeks, 2025).

El estudio de juegos adversarios ha impulsado el desarrollo de algoritmos de búsqueda capaces de explorar grandes espacios de estados y seleccionar movimientos estratégicos. Entre los enfoques más importantes destacan Minimax, Alpha-Beta Pruning y Monte Carlo Tree Search (MCTS), los cuales buscan estimar las mejores decisiones posibles mediante exploración sistemática o simulaciones probabilísticas. Sin embargo, uno de los principales desafíos de estos problemas es la explosión combinatoria, ya que la cantidad de estados posibles crece exponencialmente con la profundidad de búsqueda. Por esta razón, las heurísticas de evaluación y las técnicas de optimización resultan fundamentales para construir agentes competitivos en juegos complejos como Othello (GeeksForGeeks, 2025).

B. Algoritmo Minimax

El algoritmo Minimax es uno de los métodos fundamentales utilizados en inteligencia artificial para resolver juegos adversarios deterministas de información perfecta. Su funcionamiento se basa en representar el juego como un árbol de búsqueda donde cada nodo corresponde a un posible estado del tablero y cada nivel representa una decisión tomada por alguno de los jugadores. El objetivo principal del algoritmo es determinar la mejor jugada posible asumiendo que el oponente actuará siempre de manera óptima (Trainer, 2025).

Dentro del árbol de búsqueda existen dos tipos de nodos: los nodos MAX y los nodos MIN. Los nodos MAX representan los turnos del agente inteligente, el cual busca maximizar la utilidad o puntuación del estado del juego. Por otro lado, los nodos MIN representan los turnos del adversario, quien intenta minimizar dicha utilidad. El algoritmo evalúa recursivamente

las posibles jugadas hasta alcanzar una profundidad determinada o un estado terminal, propagando luego los valores obtenidos hacia arriba en el árbol para seleccionar la mejor decisión posible (Trainer, 2025).

Debido a que en juegos complejos como Othello no es computacionalmente viable explorar el árbol completo de decisiones, Minimax suele combinarse con funciones heurísticas de evaluación. Estas heurísticas permiten estimar la calidad de estados no terminales considerando factores estratégicos como movilidad, control de esquinas, estabilidad de fichas y ventaja posicional. Aunque Minimax garantiza decisiones óptimas bajo el supuesto de un oponente perfecto, su principal limitación es el alto costo computacional asociado al crecimiento exponencial del árbol de búsqueda, lo que se conoce como explosión combinatoria (Algorithms, s.f.).

C. Poda Alpha-Beta

La poda Alpha-Beta es una optimización del algoritmo Minimax diseñada para reducir la cantidad de nodos explorados durante la búsqueda. Su principal objetivo es eliminar ramas del árbol que no influirán en la decisión final, permitiendo alcanzar mayores profundidades de análisis con un costo computacional considerablemente menor. Gracias a esta mejora, Alpha-Beta se ha convertido en uno de los algoritmos más utilizados en juegos adversarios clásicos como ajedrez, damas y Othello (Deb, 2019).

El algoritmo utiliza dos valores principales: α (alpha) y β (beta). El valor α representa la mejor puntuación que el jugador MAX puede garantizar hasta el momento, mientras que β representa la mejor puntuación que el jugador MIN puede asegurar. Durante la exploración del árbol, si se detecta que una rama no puede producir un resultado mejor que uno ya encontrado previamente, dicha rama se descarta sin necesidad de seguir evaluándola. Este proceso se conoce como poda y permite reducir significativamente el número de estados visitados (Deb, 2019).

Aunque Alpha-Beta produce exactamente la misma decisión final que Minimax, su eficiencia es mucho mayor. Esta mejora resulta especialmente importante en juegos como Othello, donde la cantidad de movimientos posibles crece rápidamente conforme avanza la partida. Además, el desempeño de Alpha-Beta depende fuertemente del orden en que se exploran las jugadas, ya que un buen ordenamiento incrementa la cantidad de podas realizadas (Deb, 2019).

D. Expectimax

El algoritmo Expectimax es una extensión probabilística de Minimax utilizada para modelar situaciones donde existe incertidumbre o donde el comportamiento del oponente no necesariamente es completamente óptimo. A diferencia de Minimax, que asume que el adversario siempre escogerá la peor jugada posible para el agente, Expectimax incorpora nodos de probabilidad que representan decisiones aleatorias o distribuciones de posibles movimientos (GeeksForGeeks, 2025).

En el árbol de búsqueda de Expectimax existen tres tipos principales de nodos: nodos MAX, nodos MIN o probabilísticos, y nodos terminales. Los nodos MAX corresponden al agente inteligente que busca maximizar la utilidad del estado, mientras que los nodos probabilísticos calculan un valor esperado considerando todas las posibles acciones del oponente junto con una probabilidad asociada a cada una. De esta forma, en lugar de seleccionar únicamente el peor escenario posible, el algoritmo estima un promedio ponderado de los resultados futuros (GeeksForGeeks, 2025).

Expectimax resulta especialmente útil en juegos con componentes aleatorios o cuando se desea modelar oponentes imperfectos. Sin embargo, en juegos deterministas de información perfecta como Othello, su desempeño puede ser inferior al de Alpha-Beta, ya que asumir distribuciones probabilísticas sobre las jugadas del adversario no siempre refleja un comportamiento competitivo real. Además, el algoritmo suele requerir una mayor cantidad de cálculos debido a que necesita evaluar múltiples posibilidades en cada nodo probabilístico, aumentando el costo computacional de la búsqueda (GeeksForGeeks, 2025).

E. Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search (MCTS) es un algoritmo de búsqueda basado en simulaciones probabilísticas utilizado para resolver problemas de toma de decisiones en juegos adversarios. A diferencia de Minimax y Alpha-Beta, que exploran sistemáticamente el árbol de juego utilizando funciones heurísticas, MCTS construye el árbol de manera dinámica a partir de múltiples simulaciones aleatorias conocidas como playouts. Este enfoque permite concentrar el esfuerzo computacional en las regiones más prometedoras del espacio de búsqueda (Swarthmore, s.f.).

El funcionamiento de MCTS se divide en 4 etapas principales (Swarthmore, s.f.):

1. **Selección:** se recorre el árbol desde la raíz utilizando una política de selección que balancea exploración y explotación.
2. **Expansión:** cuando se alcanza un nodo no completamente explorado, se agregan nuevos nodos hijos correspondientes a movimientos posibles.
3. **Simulación:** se ejecuta una partida simulada desde el nuevo nodo hasta alcanzar un estado terminal.
4. **Retropropagación:** el resultado obtenido en la simulación se propaga hacia arriba en el árbol actualizando estadísticas de victoria y visitas.

Una de las principales ventajas de MCTS es que no depende completamente de una heurística compleja para evaluar posiciones, ya que las simulaciones permiten estimar el potencial estratégico de distintos movimientos. Esto resulta particularmente útil en juegos con espacios de búsqueda muy grandes, donde explorar exhaustivamente el árbol completo es inviable. Sin embargo, el desempeño del algoritmo depende fuertemente del número de simulaciones disponibles y del tiempo computacional asignado (Swarthmore, s.f.).

III. DISEÑO EXPERIMENTAL

A. Entorno de desarrollo

El proyecto fue desarrollado completamente en Python utilizando Jupyter Notebook como entorno principal de implementación y experimentación. La elección de Python permitió integrar de manera sencilla algoritmos de búsqueda, estructuras de datos dinámicas y herramientas de visualización para el análisis experimental. Las principales bibliotecas utilizadas fueron Numpy, Matplotlib, ipywidgets, PIL (Python Imaging Library), time, math y random.

B. Implementación del motor de juego

El núcleo del sistema fue implementado mediante la clase *GameEngine*, encargada de administrar todas las reglas y algoritmos relacionados con Othello. Esta clase incluye:

- Inicialización de tablero
- Generalización de movimientos legales
- Validación de jugadas
- Aplicación de movimientos
- Volteo de fichas
- Detección de estados terminales
- Cálculo de puntuaciones
- Evaluación heurística
- Algoritmos de búsqueda adversaria

El tablero fue representado mediante una matriz de 8x8 donde 0 representa casillas vacías, 1 representa fichas negras y 2 representa fichas blancas. Además, se implementó una lista de direcciones diagonales, horizontales y verticales para verificar capturas válidas en todas las direcciones permitidas por las reglas de Othello. La arquitectura del motor permitió reutilizar el mismo estado del juego para distintos algoritmos, garantizando condiciones equivalentes durante los experimentos comparativos.

C. Representación del estado del juego

El manejo dinámico de las partidas se implementó mediante la clase *GameState*, la cual administra:

- Estado actual del tablero
- Turno del jugador
- Historial de movimientos
- Métricas de movimiento
- Detección de finalización de partida

Esta separación entre estado y motor de búsqueda permitió mantener organizada la lógica del sistema y facilitó la simulación automática de múltiples partidas consecutivas durante los torneos IA vs IA. Asimismo, el historial de movimientos fue utilizado para registrar estadísticas y analizar posteriormente el comportamiento de los algoritmos durante las partidas.

D. Función heurística de evaluación

Debido a que explorar el árbol completo de Othello resulta computacionalmente inviable, se implementó una función

heurística para evaluar estados no terminales del juego. La heurística utilizada combina distintos factores estratégicos importantes en Othello:

- a. **Pesos posicionales:** se implementó una matriz clásica de pesos estratégicos donde las esquinas poseen valores altamente positivos y las posiciones adyacentes a las esquinas poseen penalizaciones importantes. Esto se debe a que las esquinas son posiciones estables que no pueden ser capturadas posteriormente.
- b. **Movilidad:** la movilidad mide la cantidad de movimientos legales disponibles para un jugador. Tener mayor movilidad permite controlar el ritmo estratégico de la partida y limitar las opciones del oponente.
- c. **Diferencia de fichas:** Se consideró la diferencia total entre fichas propias y fichas enemigas presentes en el tablero.
- d. **Control de esquinas:** el control de esquinas fue tratado como uno de los factores más importantes de la evaluación debido a su enorme impacto estratégico en Othello.

La combinación de estos elementos permitió construir una heurística balanceada capaz de mejorar significativamente la calidad de juego de los agentes deterministas.

E. Algoritmos implementados

El sistema incorpora tres algoritmos principales de inteligencia artificial:

- a. **Alpha-Beta Pruning:** se implementó Minimax optimizado mediante poda Alpha-Beta con profundidad configurable. El algoritmo utiliza la función heurística para evaluar estados intermedios y reducir el número de nodos explorados.
- b. **Expectimax:** se desarrolló una variante probabilística basada en valores esperados para modelar comportamientos menos deterministas del adversario.
- c. **Monte Carlo Tree Search (MCTS):** el algoritmo de MCTS fue implementado utilizando selección basada en UCT, expansión dinámica de nodos, simulaciones aleatorias y retropropagación de resultados. La cantidad de iteraciones y simulaciones fue configurable para ajustar el costo computacional del algoritmo.

F. Interfaz visual e interacción

La interfaz gráfica fue desarrollada utilizando *ipywidgets*, permitiendo ejecutar el juego directamente desde el notebook. La interfaz incluye:

- Visualización del tablero en tiempo real
- Indicación de movimientos válidos

- Selección de algoritmos para cada jugador
- Modo humano vs humano
- Modo humano vs IA
- Métricas de rendimiento por movimiento
- Reinicio automático de partidas

Cada casilla del tablero fue implementada como un botón interactivo que actualiza dinámicamente el estado visual de la partida. Además, se incorporaron indicadores de tiempo por movimiento, cantidad de nodos explorados, evaluación de heurística del tablero, resultados de torneo.

G. Diseño de los experimentos

Con el objetivo de comparar el rendimiento de los algoritmos implementados, se realizaron múltiples experimentos automáticos entre agentes inteligentes. Los principales enfrentamientos fueron:

- Alpha-Beta vs Expectimax
- Alpha-Beta vs MCTS
- MCTS vs Random

Cada torneo consistió en aproximadamente 20 partidas automáticas, alternando el color inicial de los jugadores para evitar ventajas sistemáticas asociadas al primer turno. Durante cada partida se registraron las siguientes métricas:

- Número de victorias
- Derrotas y empates
- Diferencia final de fichas
- Tiempo promedio por movimiento
- Cantidad de nodos explorados
- Duración total de la partida

Los experimentos fueron ejecutados bajo restricciones temporales similares para garantizar una comparación justa entre algoritmos.

H. Análisis de rendimiento

El notebook incorpora una sección específica de análisis experimental donde se generan automáticamente distintas gráficas de rendimiento. Entre las principales visualizaciones implementadas se encuentran:

- Explosión combinatoria:** comparación entre Minimax y Alpha-Beta respecto al crecimiento de número de nodos explorados conforme aumenta la profundidad de búsqueda.
- Factor de ramificación efectivo:** cálculo del factor de ramificación efectivo utilizando la relación entre profundidad y cantidad de nodos visitados.
- Mapas heurísticos:** visualización de pesos posicionales utilizados por la heurística del agente.
- Tiempos y nodos por movimiento:** análisis del costo

computacional real durante partidas completas.

Estas gráficas permitieron evaluar tanto la eficiencia computacional como la calidad estratégica de los algoritmos implementados.

IV. RESULTADOS

En esta sección se presentan los resultados obtenidos de la implementación de los algoritmos de búsqueda adversaria para el juego de Othello. Se analizarán la eficiencia computacional, el rendimiento de cada algoritmo y la efectividad de la heurística implementada, todo esto a través de torneos simulados entre los agentes, así como las métricas de nodos y tiempo. Las figuras permiten visualizar los resultados obtenidos en distintas fases del juego.

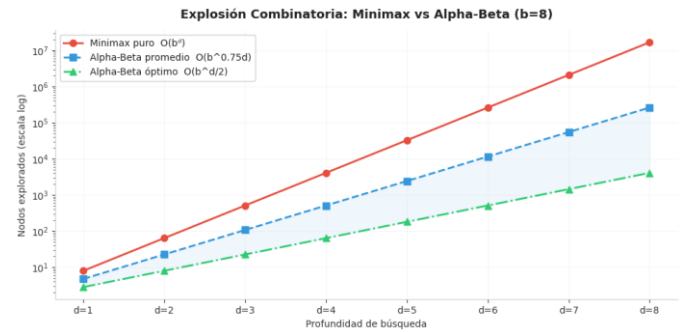


Figura 1 - Nodos explorados Minimax vs Alpha-Beta

Como se observa en la Figura 1, esta representa los nodos explorados y la profundidad de búsqueda en el primer caso de uso, que sería Minimax vs Alpha-Beta. Como se puede observar la cantidad de nodos explorados crece de manera exponencial para el caso de Minimax, en el caso de Alpha-Beta se puede observar que los nodos explorados se reducen gracias a la poda del árbol que este realiza. Para analizar de mejor manera la poda se realizó un factor de ramificación.

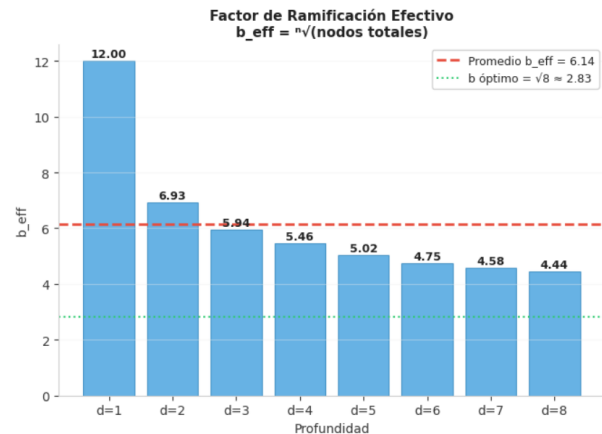


Figura 2 - Factor de ramificación

La Figura 2 representa el factor de ramificación en relación con los nodos totales y la profundidad del árbol. En esta se puede observar que el factor experimental tiene una media de 6.14, lo cual se acerca al valor teórico esperado que en este

caso sería de 2.83, indicando la mejora en eficiencia que proporciona la poda comparada con otros algoritmos.

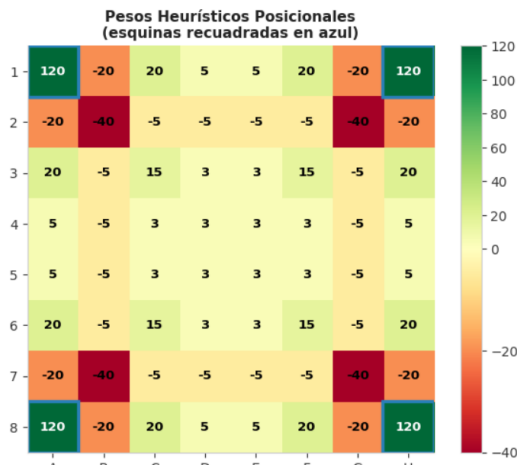


Figura 3 - Pesos heurísticos

La figura 3 muestra la matriz de pesos heurísticos posicionales, es decir los pesos en cada celda del tablero. Como se puede observar las esquinas se priorizan con un valor de 120 y las casillas adyacentes a las esquinas representan una penalización importante, esto se debe a que estas posiciones son de vital importancia en Othello y por lo mismo la heurística en estos puntos es más importante. Adicional a esto se puede observar que en el centro del tablero los pesos de heurísticas son bajos.

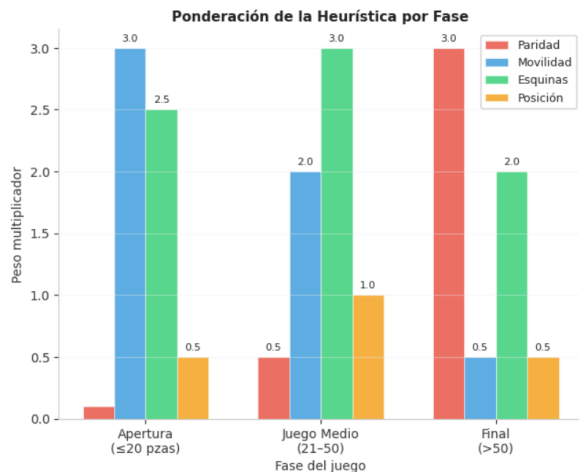


Figura 4 - Ponderación de Heurística por fase

La Figura 4 representa la ponderación de la heurística por fase de juego, ya que en cada fase la prioridad es diferente. En la fase de apertura la movilidad y las esquinas son el factor más importantes, en el juego medio gana aun más prioridad las esquinas y ya en el final del juego la paridad se queda con máxima prioridad. Como se puede ver en cada fase la posición es lo menos importante y las esquinas son clave para la resolución del juego ya que en todas las fases es importante para la heurística.

A continuación, se muestran los resultados de los torneos realizados entre agentes.

#	Ganador	Mostr	U/mov	U/mov 2
#1	● Negro	36 - 28	60 mov.	280 ms
#2	● Negro	36 - 28	60 mov.	199 ms
#3	● Negro	36 - 28	60 mov.	180 ms
#4	● Negro	36 - 28	60 mov.	170 ms
#5	● Negro	36 - 28	60 mov.	160 ms
#6	● Negro	36 - 28	60 mov.	160 ms
#7	● Negro	36 - 28	60 mov.	177 ms
#8	● Negro	36 - 28	60 mov.	206 ms
#9	● Negro	36 - 28	60 mov.	206 ms
#10	● Negro	36 - 28	60 mov.	184 ms

Figura 5 - Resultados Alpha-Beta vs Expectimax

En el torneo de Alpha-Beta vs Expectimax, se muestra en la Figura 5, Alpha-Beta ganó todas las partidas con una diferencia promedio de fichas de 36 contra 28. También se puede observar el tiempo de cada algoritmo, donde Alpha-Beta se mantiene más rápido con aproximadamente 185ms frente a 330ms de Expectimax. Estos resultados evidencian que la poda y la función de heurística proporciona movimientos más consistentes y eficientes, mientras que Expectimax, al incorporar decisiones probabilísticas genera movimientos más lentos y menos efectivos.

#	Ganador	Mostr	U/mov	U/mov 2
#1	○ Blanco	28 - 40	60 mov.	350 ms
#2	● Negro	33 - 31	60 mov.	311 ms
#3	● Negro	36 - 28	60 mov.	275 ms
#4	○ Blanco	22 - 42	60 mov.	280 ms
#5	● Negro	40 - 24	60 mov.	197 ms
#6	○ Blanco	26 - 38	60 mov.	233 ms
#7	○ Blanco	31 - 33	60 mov.	233 ms
#8	○ Blanco	24 - 40	60 mov.	206 ms
#9	○ Blanco	9 - 55	60 mov.	261 ms
#10	○ Blanco	31 - 33	60 mov.	169 ms
#11	○ Blanco	36 - 34	60 mov.	275 ms
#12	○ Blanco	21 - 43	60 mov.	238 ms
#13	○ Blanco	25 - 39	60 mov.	208 ms
#14	● Negro	39 - 25	60 mov.	163 ms
#15	○ Blanco	21 - 42	59 mov.	232 ms
#16	○ Blanco	20 - 44	60 mov.	207 ms
#17	○ Blanco	19 - 45	60 mov.	252 ms
#18	○ Blanco	17 - 47	60 mov.	246 ms
#19	○ Blanco	26 - 38	60 mov.	324 ms
#20	● Negro	43 - 21	60 mov.	399 ms

Figura 6 - Resultados Alpha-Beta vs MCST

Para el torneo Alpha-Beta vs MCTS, la Figura 6 muestra los resultados de las 20 partidas, donde MCTS ganó quince y Alpha-Beta cinco, con diferencias promedios de fichas de 34 a favor de MCTS frente a 30 de Alpha-Beta.



Figura 7 - Tiempo y Nodos explorados Alpha-Beta vs MCTS

Para este torneo se realizaron graficas para comparar los tiempos de jugadas y los nodos explorados, destacando que MCTS explora estratégicamente las jugadas y se adapta a oponentes complejos, aunque con mayor tiempo de cálculo

por movimiento, mientras que Alpha-Beta mantiene consistencia y rapidez.

#	Estado	● Negro	● Blanco	● Verde	● Azul	● Naranja	● Púrpura	● Gris	● Negro
#1	● Negro	47	17	60	60	60	60	60	0
#2	● Negro	34	20	60	60	60	60	60	0
#3	● Negro	40	24	60	60	60	60	60	0
#4	● Negro	42	22	60	60	60	60	60	0
#5	● Negro	38	26	60	60	60	60	60	0
#6	● Negro	46	18	60	60	60	60	60	0
#7	● Negro	51	13	60	60	60	60	60	0
#8	● Negro	37	27	60	60	60	60	60	0
#9	● Negro	38	26	60	60	60	60	60	0
#10	● Negro	54	10	60	60	60	60	60	0

Figura 8 - Resultados MCTS vs Random

Finalmente, en el torneo MCTS vs Random, la Figura 8 muestra los resultados donde MCTS ganó diez partidas con una diferencia promedio de fichas de 47 a 17. Como se puede observar en los tiempos MCTS es más eficiente frente a un agente sin estrategia demostrando su robustez.

La comparación general de los algoritmos indica que Alpha-Beta ofrece decisiones óptimas y consistentes, siendo ideal frente a oponentes deterministas, mientras que MCTS sobresale en exploración estratégica y adaptabilidad, aunque requiere más tiempo de cálculo. Expectimax puede ser útil para oponentes con comportamiento probabilístico, pero resulta menos efectivo frente a Alpha-Beta, y el agente aleatorio se comporta como referencia de desempeño mínimo.

V. DISCUSIÓN DE RESULTADOS

Los resultados obtenidos confirman la importancia de las técnicas de poda en los algoritmos de búsqueda adversaria. La comparación entre Minimax y Alpha-Beta mostró una reducción significativa en la cantidad de nodos explorados a medida que aumenta la profundidad de búsqueda. Aunque el factor de ramificación efectivo no alcanzó el valor teórico óptimo, los resultados evidencian que Alpha-Beta mejora considerablemente la eficiencia computacional sin afectar la calidad de las decisiones generadas.

La función heurística implementada también tuvo un impacto importante en el desempeño de los agentes. La priorización de las esquinas y la penalización de las casillas adyacentes muestran principios estratégicos fundamentales de Othello, mientras que la ponderación dinámica según la fase del juego permitió adaptar la evaluación a diferentes situaciones. Esto contribuyó a que Alpha-Beta realizara movimientos más consistentes y mantuviera un buen rendimiento en los torneos realizados.

En el enfrentamiento entre Alpha-Beta y Expectimax, Alpha-Beta obtuvo la totalidad de las victorias y presentó menores tiempos de ejecución. Este comportamiento puede explicarse porque Othello es un juego determinista, por lo que las ventajas teóricas de Expectimax para entornos con incertidumbre no logran traducirse en mejores resultados. En consecuencia, la incorporación de cálculos probabilísticos incrementó el costo computacional sin generar beneficios competitivos significativos.

Por otro lado, el torneo entre Alpha-Beta y MCTS mostró que Monte Carlo Tree Search posee una mayor capacidad de exploración estratégica, obteniendo la mayoría de las victorias. Aunque sus tiempos de cálculo fueron superiores, las simulaciones realizadas permitieron identificar secuencias de juego favorables que no siempre son detectadas mediante búsqueda heurística tradicional. De la misma manera, la amplia ventaja obtenida por MCTS frente al agente aleatorio confirmó la efectividad de este enfoque para desenvolverse en espacios de búsqueda complejos.

De manera general, los experimentos demuestran que cada algoritmo posee fortalezas particulares. Alpha-Beta destaca por su rapidez y consistencia cuando dispone de una función heurística adecuada, mientras que MCTS ofrece un mejor desempeño competitivo gracias a su capacidad exploratoria. Los resultados indican que la elección del algoritmo depende del equilibrio deseado entre calidad estratégica, tiempo de respuesta y recursos computacionales disponibles.

VI. CONCLUSIONES

- Alpha-Beta redujo significativamente la cantidad de nodos explorados respecto a Minimax, demostrando que la poda mejora la eficiencia computacional sin afectar la calidad de las decisiones.
- La heurística basada en pesos posicionales y ponderación por fases contribuyó de manera importante al desempeño de los agentes, validando la relevancia del conocimiento estratégico dentro del proceso de evaluación.
- Alpha-Beta obtuvo mejores resultados que Expectimax tanto en victorias como en tiempo de ejecución, evidenciando que los algoritmos deterministas son más adecuados para Othello que aquellos diseñados para escenarios probabilísticos.
- Monte Carlo Tree Search fue el agente más competitivo de los experimentos, obteniendo la mayor cantidad de victorias en los torneos realizados, aunque con mayores tiempos de cálculo por movimiento que Alpha-Beta.
- El sistema desarrollado cumplió con el objetivo de implementar y comparar distintos algoritmos de búsqueda adversaria, permitiendo identificar sus ventajas, limitaciones y aplicaciones dentro del contexto del juego Othello.

VII. RECOMENDACIONES

Se recomienda mejorar el algoritmo Alpha-Beta mediante técnicas de ordenamiento de movimientos, ya que el factor de ramificación efectivo obtenido indica que aún existe margen para incrementar la efectividad de la poda. Por otro lado, se recomienda ampliar la función heurística incorporando factores estratégicos adicionales, como estabilidad de fichas o con-

trol de bordes, para evaluar si es posible mejorar el desempeño competitivo del agente. Finalmente, sería conveniente realizar más experimentos con diferentes configuraciones de simulaciones en MCTS y una mayor cantidad de partidas por torneo, con el fin de obtener resultados más robustos y analizar con mayor precisión las fortalezas y limitaciones de cada algoritmo.

BIBLIOGRAFÍA

- [1] Algorithms. (s. f.). <https://cs.stanford.edu/people/eroberts/courses/soco/projects/2003-04/intelligent-search/minimax.html>
- [2] Deb, Sayantini (2019, 12 septiembre). What is Alpha Beta Pruning in Artificial Intelligence? Medium. <https://medium.com/edureka/alpha-beta-pruning-in-ai-b47ee5500f9a>
- [3] First Trainer (2025, 2 junio). Mini-Max Algorithm in Artificial Intelligence. Medium. <https://medium.com/@mhassaan.1/mini-max-algorithm-in-artificial-intelligence-e0a0da694b3b>
- [4] GeeksforGeeks. (2025, 23 julio). Adversarial Search Algorithms in Artificial Intelligence (AI). GeeksforGeeks. <https://www.geeksforgeeks.org/artificial-intelligence/adversarial-search-algorithms/>
- [5] GeeksforGeeks. (2025b, julio 23). Expectimax Search Algorithm in AI. GeeksforGeeks. <https://www.geeksforgeeks.org/artificial-intelligence/expectimax-search-algorithm-in-ai/>
- [6] Monte Carlo Tree Search - about. (s. f.). <https://www.cs.swarthmore.edu/~mitchell/classes/cs63/f20/reading/mcts.html>